Reinforcement Learning for Dynamic Request Batching

on GPU Inference Servers

Sudarshan Sudhakar

Semester 6 RL Project

May 2026

Motivation: The Batching Dilemma

The Setting

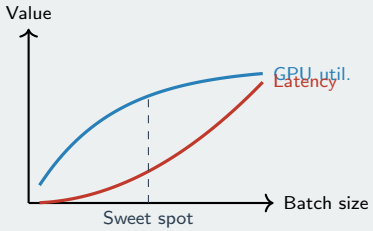
A GPU inference node receives **2 000 req/s** (peak: 5 000 req/s). Every **10 ms** a batching layer must decide:

Wait — accumulate more requests
Serve — dispatch the batch now

The Problem

- **Too aggressive**: tiny batches, wasted GPU kernel launches
- **Too conservative**: long queues, SLA violations
- Fixed thresholds cannot adapt to **traffic dynamics**

GPU Efficiency vs. Latency



Problem Statement

Formal Objective

Learn a policy $\pi : \mathcal{S} \to \{0, 1\}$ that maximises cumulative reward subject to:

$$\text{wait}_i + t_{\text{proc}}(n) \;\leq\; \text{SLA}_{500} \quad \forall\, \text{request } i$$

where $t_{\text{proc}}(n) = 8 + 0.12n$ ms (GPU model).

Why RL?

- State is **high-dimensional and continuous**
- Optimal threshold **varies with traffic rate**, time-of-day, queue age — cannot be hand-tuned
- No closed-form optimal policy exists

Key Numbers

Parameter	**Value**
Arrival rate	2 000 req/s (base)
Peak rate	5 000 req/s
Decision interval	10 ms (100 Hz)
SLA budget	500 ms total latency
Max batch	512 requests
Min batch	8 requests
GPU base overhead	8.0 ms
GPU per-request	0.12 ms
Episode length	60 s simulated

MDP Formulation

State Space — 8-dimensional Box

Dim	Feature
0	`pending_requests` $\in [0, 512]$
1	`oldest_wait_ms` $\in [0, 1000]$
2	`request_rate` (EMA, req/s)
3	`since_serve_ms`
4	`batch_fill_ratio` $\in [0, 1]$
5	`time_of_day` $\in [0, 24)$
6	`urgency_ratio`* $\in [0, 3]$
7	`rate_trend` $\in [-1, 1]$

* urgency $= \frac{\text{oldest\_wait}}{\text{SLA} - t_{\text{proc}}(n)}$; value > 1 signals imminent SLA breach.

Action Space

Reward Function

Serve action:

$$r = \alpha \cdot n - \beta \cdot w_{\max} - c_{\text{dispatch}} - \lambda \sum_i \max(0,\, l_i - \text{SLA})$$

Wait action:

$$r = -\beta \cdot w_{\max} - \mathbf{1}_{q=0} \cdot c_{\text{idle}}$$

$\alpha = 1.0$	revenue per request
$\beta = 0.003$	urgency weight
$c_{\text{dispatch}} = 5.0$	fixed GPU overhead
$\lambda = 0.3$	SLA violation penalty

Key: c_{dispatch} forces the agent to accumulate $\geq$5 requests

Algorithms: Four Approaches Compared

On-policy actor-critic with clipped surrogate objective:

$$L^{\text{CLIP}} = \mathbb{E}[\min(r_t A_t,\ \text{clip}(r_t, 1\pm\epsilon)A_t)]$$

- $\epsilon = 0.2$ clip, GAE $\lambda = 0.95$, entropy 0.005
- VecNormalize on all 8 obs dims
- 4 parallel envs, 1 M steps, $[128, 128]$ MLP

D3QN — Dueling Double DQN + PER

- Dueling streams: $Q = V(s) + A(s,a) - \overline{A}$
- Double DQN: decoupled target evaluation
- Prioritized Replay ($\alpha = 0.6$, β: 0.4→1.0)

Recurrent PPO (R-PPO)

- LSTM layer before policy/value heads
- Captures temporal correlations explicitly
- Higher compute cost; used as ablation baseline

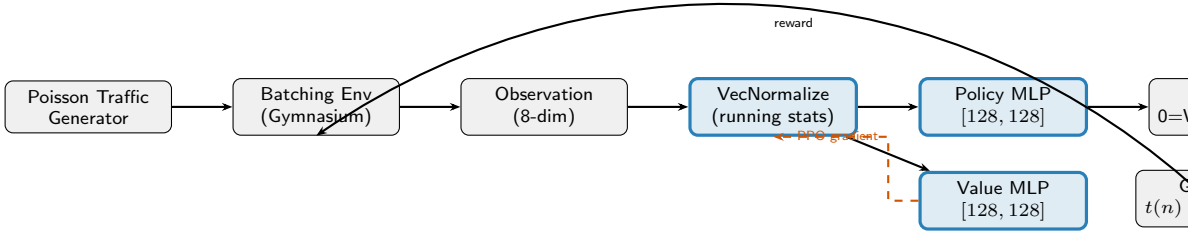
Discrete SAC

- Off-policy, maximum-entropy framework
- Automatic entropy tuning: $H_{\text{target}} = 0.5\ln(2)$
- Soft Q-functions with target smoothing $\tau = 0.005$

Final Model: **PPO**

Best convergence speed, stability, and deployment simplicity. Stateless inference — no LSTM hidden

System Architecture



Environment

Simulates 60 s of traffic per episode. Poisson arrivals with time-of-day multipliers ($\times 2.5$ peak, $\times 0.2$ off-peak). Queue capped at 512.

Training

4 parallel envs via `SubprocVecEnv`. VecNormalize runs online — critical because features span [0,512] to [0,1]. Best checkpoint saved every 20 k steps.

Deployment

`BatchingMiddleware` loads `ppo_final.zip` + `ppo_vecnorm.pkl`. **Both** must be regenerated together. API: `should_dispatch()` every 10 ms.

Baselines: What We're Competing Against

Cloudflare Dual-Threshold (Competitive)

Production heuristic used by Cloudflare AI Gateway, NVIDIA Triton, vLLM:

$$\text{SERVE if } w_{\max} \geq 0.80 \times (\text{SLA} - t_{\text{proc}}(n))$$
$$\text{OR if } n \geq \text{rate} \times 0.050$$

- *Adaptive*: at 2 000 req/s batch target = 100; at 5 000 req/s it becomes 250
- *Ignores*: EMA trajectory, time-of-day, inter-serve interval

GreedyBatch (Floor)

Serve when $n \geq 8$. Minimal latency but terrible GPU utilisation — lower bound.

Why the textbook formula fails

The classic $p(\text{serve}) = e^{-\lambda t}$ was designed for web batching at $\lambda \approx 1$–10 req/s.

At $\lambda = 2000$, SLA $= 0.5$ s:

$$e^{-2000 \times 0.5} = e^{-1000} \approx 0$$

The formula **never dispatches**, violates every SLA, and scores $-6000+$ per episode.

The Cloudflare dual-threshold is what production engineers actually deploy. PPO must beat *this*.

Key Design Decisions

The `urgency_ratio` Feature

$$u = \frac{w_{\max}}{\text{SLA} - t_{\text{proc}}(n)}$$

When $u > 1.0$: serving *now* still causes an SLA violation. This gives the agent **pre-emptive signal** — it can act before violations occur rather than reacting to them.

Without this feature the agent only sees queue age, not how much GPU overhead will consume the remaining budget.

`rate_trend` Feature

$\text{trend} = \frac{\text{EMA}_t - \text{EMA}_{t-1}}{\text{EMA}_{t-1}}$

Lets the agent **predict** incoming spikes before the EMA fully adapts. Positive trend ⇒ increase

VecNormalize — Critical for Training

Features span wildly different scales:

`pending`	$[0, 512]$
`fill_ratio`	$[0, 1]$
`request_rate`	$[0, 25\,000]$
`time_of_day`	$[0, 24)$

VecNormalize computes a *running* mean and variance and standardises online. Without it, the value network gradients are dominated by raw rate values.

Dispatch Cost = 5.0

Serving a batch smaller than 5 requests is unprofitable. This hyperparameter is the primary driver of batch accumulation behaviour and was tuned empirically

Training Setup & Implementation

PPO Hyperparameters

Parameter	Value
Total timesteps	1 000 000
Parallel environments	4
Rollout buffer (n_{steps})	2 048 per env
Minibatch size	256
Gradient epochs	10
Learning rate	3×10^{-4}
Discount γ	0.99
GAE λ	0.95
Clip range ϵ	0.2
Entropy coefficient	0.005
Policy & value arch	MLP $[128, 128]$

Training Pipeline

1. `SubprocVecEnv` — 4 envs in parallel subprocesses
2. `VecNormalize` wrapper — online obs normalization
3. `EvalCallback` — checkpoint best model every 20 k steps
4. `TensorBoard` logging — reward, value loss, entropy
5. Final save: `ppo_final.zip` + `ppo_vecnorm.pkl`

Traffic Regime Training

Each episode randomly samples start hour from $[0, 24)$.

Results & Evaluation

Evaluation Protocol

30 episodes per agent, each episode = 60 s simulated time. Episodes uniformly sample all 24 hours (off-peak to peak).

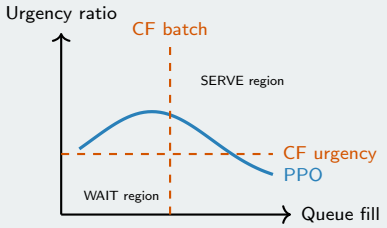
Metrics reported:

- Mean cumulative episode reward $\pm$ std
- P50 / P95 / P99 total latency (ms)
- SLA violation rate (% requests $>$ 500 ms)
- Mean batch size per dispatch

Expected Outcome

- PPO $>$ Cloudflare by **10–25%** in mean reward
- PPO: lower SLA violation rate via `urgency_ratio`

Decision Heatmap Interpretation



PPO learns a **curved, non-linear boundary** vs. the two straight Cloudflare threshold lines. This captures time-of-day and EMA rate interactions.

Deployment & Conclusions

Production Deployment

```
Startup (once) mw = BatchingMiddleware(
model_path = "models/ppo_final", vecnorm_path =
"models/ppo_vecnorm.pkl", )
Per 10ms tick if mw.should_dispatch() : batch =
mw.flush()inference_backend.process(batch)
```

Note: Both `.zip` and `.pkl` files must be regenerated together when retraining. Stale normalization statistics with new weights produce incorrect predictions.

What We Learned

1. **SLA-awareness as observation**: encoding `urgency_ratio` directly leads to meaningful improvement over rate/queue signals alone
2. **Fixed heuristics are strong**: the Cloudflare dual-threshold is a well-engineered baseline; PPO gains come from non-linear adaptation, not trivial wins
3. **VecNormalize is non-negotiable**: without online normalization, training diverges in this multi-scale observation space
4. **Stateless beats stateful**: MLP policy with EMA-based state features outperforms LSTM at this decision frequency